

# Support langage C

# Sommaire

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>Partie 1 : Initiation au langage C : Notions de base .....</b>   | <b>3</b>  |
| <b>Partie 2 : Les fonctions d'E/S standards.....</b>                | <b>10</b> |
| <b>Partie 3 : Les instructions de branchement conditionnel.....</b> | <b>13</b> |
| <b>Partie 4 : Les structures répétitives.....</b>                   | <b>15</b> |
| <b>Partie 5 : Les Tableaux.....</b>                                 | <b>17</b> |
| <b>Partie 6 : Les chaînes de caractères.....</b>                    | <b>20</b> |
| <b>Partie 7 : Les Fonctions.....</b>                                | <b>22</b> |
| <b>Partie 8 : Les Structures.....</b>                               | <b>25</b> |
| <b>Partie 9 : Pointeurs et Allocation dynamique.....</b>            | <b>30</b> |
| <b>Partie 10 : Les listes chaînées.....</b>                         | <b>34</b> |

# Partie 1 : Initiation au langage C : Notions de base

## 1. Introduction au langage C

### 1.1. Historique

Le C a été conçu en 1972 par Dennis Richie et Ken Thompson chercheurs aux Bell Labs afin de développer un système d'exploitation UNIX sur un DEC PDP-11. En 1978 Brian Kernighan et Dennis Richie publient la définition classique du C dans le livre « The C Programming language ». Le C devenant de plus en plus populaire dans les années 80, plusieurs groupes mirent sur le marché des compilateurs comportant des extensions particulières. En 1983, l'ANSI (American National Standards Institute) décida de normaliser le langage ; ce travail s'acheva en 1989 par la définition de la norme ANSI-C. Celle-ci fut reprise telle qu'elle par l'ISO (International Standards Organization).

### 1.2. Présentation générale du langage C

Le langage C possède assez peu d'instructions, il fait par contre appel à des bibliothèques, fournies en plus ou moins grand nombre avec le compilateur.

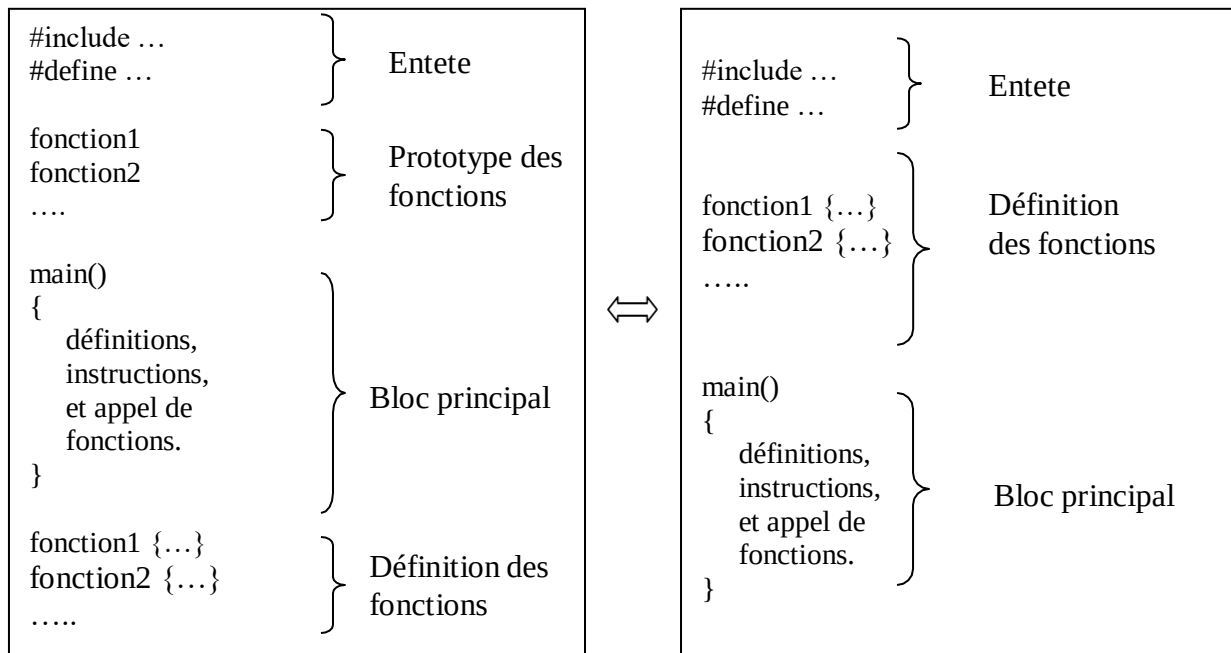
Exemples:    math.h : bibliothèque de fonctions mathématiques  
                  stdio.h : bibliothèque d'entrées/sorties standard

On ne saurait développer un programme en C sans se munir de la documentation concernant ces bibliothèques.

## 2. Structure d'un programme C

Un programme en C comporte :

- **Un entête** (header) constitué de méta instructions ou **directives** destinées au pré processeur.
- **Un bloc principal** appelé **main ()**. Il contient des instructions élémentaires ainsi que les appels aux modules (fonctions) considérés par le compilateur comme des entités autonomes. Chaque appel à une fonction peut être considéré comme une instruction.
- **Le corps** des fonctions placées avant ou après le main () dans un ordre quelconque, les une après les autres. Partout, les variables et les fonctions font l'objet d'une déclaration précisant leur type. Le schéma général est donc :



**Figure : Structure d'un programme C**

Exemple : calcul de la surface d'un cercle.

```
#include <stdio.h>
#define PI 3.14
float f_surface(float rayon)
{float s;
  s=rayon*rayon*PI;
  return(s);
}
void main()
{ float surface;
  surface=f_surface(2.0);
  printf("%f\n",surface);
}
```

```
#include <stdio.h>
#define PI 3.14
float f_surface(float);
void main()
{
  float surface;
  surface=f_surface(2.0);
  printf("%f\n",surface);
}
float f_surface(float rayon)
{float s;
  s=rayon*rayon*PI;
  return(s);
}
```

### 3. Les composants élémentaires du C

Un programme en langage C est constitué des six groupes de composants élémentaires suivants :

- les identificateurs
- les mots-clefs
- les constantes
- les chaînes de caractères,
- les opérateurs
- les signes de ponctuation.

On peut ajouter à ces six groupes les commentaires, qui ne sont pas considérés par le pré processeur.

### 3.1. Les identificateurs

- Le rôle d'un identificateur est de donner un nom à une entité du programme. Plus précisément, un identificateur peut désigner :
  - un nom de variable ou de fonction,
  - un type défini par typedef, struct, union ou enum,
  - une étiquette.
- Un identificateur est une suite de caractères parmi les lettres (minuscules ou majuscules, mais non accentuées), les chiffres, le tiret bas.
- Le premier caractère d'un identificateur ne peut pas être un chiffre.
- Les majuscules et minuscules sont différenciées.
- Le compilateur peut tronquer les identificateurs au-delà d'une certaine longueur. Cette limite dépend des implémentations, mais elle est toujours supérieure à 31 caractères. (Le standard dit que les identificateurs externes, c'est-à-dire ceux qui sont exportés à l'édition de lien, peuvent être tronqués à 6 caractères, mais tous les compilateurs modernes distinguent au moins 31 caractères).

### 3.2. Les mots-clefs

Un certain nombre de mots, appelés mots-clefs, sont réservés pour le langage lui-même et ne peuvent pas être utilisés comme identificateurs. L'ANSI-C compte 32 mots clefs :

**auto const double float int short struct unsigned  
break continue else for long signed switch void  
case default enum goto register sizeof typedef volatile  
char do extern if return static union while**

### 3.3. Les commentaires

Un commentaire débute par `/*` et se termine par `*/`. Par exemple : `/* Ceci est un commentaire */`. On ne peut pas imbriquer des commentaires.

### 3.4. Les constantes

Une constante est une valeur qui apparaît littéralement dans le code source d'un programme, le type de la constante étant déterminé par la façon dont la constante est écrite. Les constantes vont être utilisées, par exemple, pour l'initialisation de variables.

Exemples :

```
int NB_ligne ;      /* déclaration d'une variable de type entière */
NB_ligne = 15 ;     /* initialisation de la variable à 15 */
const annee = 2022 /* affecte une valeur à une variable, cette valeur ne pourra plus être changée */
```

## 4. Les types de données

En C, il n'existe que quelques types fondamentaux de données :

- Le type **char** : un seul byte représentant un caractère.
- Le type **int** : un nombre entier dont la taille correspond à celles des entiers du SE.
- Le type **float** : un nombre en virgule flottante en simple précision.
- Le type **double** : un nombre en virgule flottante en double précision.

## 5. Déclaration de variables

Les variables doivent toutes être déclarées avant d'être utilisées bien que certaines soient faites implicitement par le contexte. A une variable correspond un espace en mémoire et un mécanisme permettant d'adresser cet emplacement. En C une variable est caractérisée par :

- Son nom (un identificateur)
- Son type (type de base ou type définie par l'utilisateur)

Une déclaration de variables a la forme :

|                                            |
|--------------------------------------------|
| <b>type</b> <b>liste d'identificateurs</b> |
|--------------------------------------------|

Exemples :

|                                     |
|-------------------------------------|
| <i>int x ;</i><br><i>double z ;</i> |
|-------------------------------------|

Une déclaration indique le nom, le type (parfois la valeur) et regroupe derrière une ou plusieurs variables de même type.

On peut initialiser les variables lors de la déclaration :

Exemples :

|                    |
|--------------------|
| <i>int i = 0 ;</i> |
|--------------------|

## 6. Les opérateurs

### 6.1. L'affectation

En C, l'affectation est un opérateur à part entière. Elle est symbolisée par le signe =. Sa syntaxe est la suivante :

|                                |
|--------------------------------|
| <b>variable = expression ;</b> |
|--------------------------------|

Le terme de gauche de l'affectation peut être une variable simple, un élément de tableau, ...etc. Cette expression a pour effet d'évaluer « expression » et d'affecter la valeur obtenue à « variable ». De plus, cette expression possède une valeur, qui est celle de « expression ». Ainsi, l'expression `i = 5` vaut 5.

L'affectation effectue une conversion de type implicite : la valeur de l'expression (terme de droite) est convertie dans le type du terme de gauche. Par exemple, le programme suivant :

|                                                                                                       |
|-------------------------------------------------------------------------------------------------------|
| <pre>main() { int i, j = 2 ;   float x = 2.5 ;   i = j + x ;   x = x + i ;   printf(“%f”,x) ; }</pre> |
|-------------------------------------------------------------------------------------------------------|

imprime pour x la valeur 6.5 (et non 7), car dans l'instruction  $i = j + x$ , l'expression  $j + x$  a été convertie en entier.

## 6.2. Les opérateurs arithmétiques

Les opérateurs arithmétiques classiques sont l'opérateur unaire - (changement de signe) ainsi que les opérateurs binaires :

+ addition ; - soustraction ; \* multiplication ; / division ;  
% reste de la division (modulo) (pour les entiers)

Ces opérateurs agissent de la façon attendue sur les entiers comme sur les flottants. Leurs seules spécificités sont les suivantes :

- Contrairement à d'autres langages, le C ne dispose que de la notation / pour désigner à la fois la division entière et la division entre flottants. Si les deux opérandes sont de type entier, l'opérateur / produira une division entière (quotient de la division). Par contre, il délivrera une valeur flottante dès que l'un des opérandes est un flottant.
- L'opérateur % ne s'applique qu'à des opérandes de type entier. Si l'un des deux opérandes est négatif, le signe du reste dépend de l'implémentation, mais il est en général le même que celui du dividende.
- Notons enfin qu'il n'y a pas en C d'opérateur effectuant l'élévation à la puissance. De façon générale, il faut utiliser la fonction `pow(x,y)` de la librairie `math.h` pour calculer  $x^y$

## 6.3. Les opérateurs relationnels

> Strictement supérieur ; >= supérieur ou égal ; < Strictement inférieur ;  
<= inférieur ou égal ; == égal ; != différent

Leur syntaxe est : `expression1 op expression2`

Les deux expressions sont évaluées puis comparées. La valeur rendue est de type `int` (il n'y a pas de type booléen en C); elle vaut 0 si la condition est fautive, et une valeur différente de 0 sinon. Attention à ne pas confondre l'opérateur de test d'égalité `==` avec l'opérateur d'affectation `=`. Ainsi, le programme suivant imprime à l'écran : a et b sont égaux .:

```
main()
{ int a = 0 ; int b =1 ;
  if (a = b)
    printf("a et b sont égaux") ;
  else
    printf("a et b sont différents") ;
}
```

## 6.4. Les opérateurs logiques (booléens)

&& : et logique      || : ou logique      ! : négation logique

Comme pour les opérateurs de comparaison, la valeur retournée par ces opérateurs est un `int` qui vaut 0 si la condition est fautive.

Dans une expression de type : `expression1 op1 expression2 op2 ...expressionN` , l'évaluation se fait de gauche à droite et s'arrête dès que le résultat final est déterminé.

## 6.5. Les opérateurs d'affectation composée

Les opérateurs d'affectation composée sont : += -= \*= /= %=

Pour tout opérateur **op**, l'expression :

expression1 **op**= expression2  $\Leftrightarrow$  expression1 = expression1 **op** expression2.

## 6.6. Les opérateurs d'incrément et de décrémentation

Les opérateurs d'incrément ++ et de décrémentation -- s'utilisent aussi bien en suffixe (i++) qu'en préfixe (++i). Dans les deux cas la variable i sera incrémentée, toutefois dans la notation suffixe la valeur retournée sera l'ancienne valeur de i alors que dans la notation préfixe ce sera la nouvelle. Par exemple :

```
int a = 3, b, c ;  
b = ++a ;      /* a et b valent 4 */  
c = b++ ;      /* c vaut 4 et b vaut 5 */
```

## 6.7. L'opérateur conditionnel ternaire

L'opérateur conditionnel ? est un opérateur ternaire. Sa syntaxe est la suivante :

*Condition ? expression1 : expression2*

Cette expression est égale à expression1 si condition est satisfaite, et à expression2 sinon. Par exemple, l'expression  $x >= 0 ? x : -x$  correspond à la valeur absolue d'un nombre. De même l'instruction `m = ((a > b) ? a : b);` affecte à m le maximum de a et de b.

## 6.8. L'opérateur de conversion de type

L'opérateur de conversion de type, appelé **cast**, permet de modifier explicitement le type d'un objet. On écrit **(type) objet** Par exemple : la valeur 1.5 sera affichée.

```
int i = 3, j = 2 ;  
printf(“%f “, (float)i/j) ;
```

## 6.9. L'opérateur adresse

L'opérateur d'adresse & appliqué à une variable retourne l'adresse mémoire de cette variable. La syntaxe est **&objet** .

# 7. Les types prédéfinis

Les types de base en C concernent les caractères, les entiers et les flottants (nombres réels). Ils sont désignés par les mots-clefs suivants :

**char int float double**

## 7.1. Les types caractères

Le mot-clef char désigne un objet de type caractère. Il est codé sur un octet et il peut être assimilé à un entier : tout objet de type char peut être utilisé dans une expression qui utilise des objets de type entier.

Exemple : le programme suivant affiche la lettre B.

```
char c = 'A';  
printf("%c", c+1);
```

## 7.2. Les types entiers

Un objet de type entier : int est représenté par un mot de 32 bits pour un PC Intel.

Le type int peut être précédé d'un attribut de précision (short ou long) et/ou d'un attribut de représentation (unsigned). Les valeurs maximales et minimales des différents types entiers sont définies dans la librairie standard limits.h. Le mot-clé sizeof a pour syntaxe sizeof(expression) où expression est un type ou un objet. Le résultat est un entier égal au nombre d'octets nécessaires pour stocker le type ou l'objet.

## 7.3. Les types flottants

Les types float, double et long double servent à représenter des nombres en virgule flottante. Ils correspondent aux différentes précisions possibles.

## Partie 2 : Les fonctions d'E/S standards

### 1. Les fonctions d'entrées

#### 1.1. La fonction scanf

Cette fonction lit à partir de l'entrée standard (clavier) une liste de variables en mémoire selon un format donné.

```
int scanf (const char *format, liste d'adresses) ;
```

**liste d'adresses** représente une liste d'adresses de variables déclarées ;

**format** représente le format de lecture des données. Elle est composée de spécificateurs qui indique le type de variables qu'on va lire. Chaque spécificateur correspond à un type de variable et doit donc apparaître dans le même ordre que la variable dans la liste. Les principaux spécificateurs de format sont résumés dans le tableau suivant :

| Spécificateur  | Signification                            |
|----------------|------------------------------------------|
| %d ou %x ou %o | Pour une variable de type int            |
| %u             | Pour une variable de type unsigned int   |
| %h             | Pour une variable de type short int      |
| %f             | Pour une variable de type float          |
| %lf            | Pour une variable de type double         |
| %c             | Pour une variable de type char           |
| %s             | Pour une variable de type texte (chaîne) |

Exemple :

```
int a = -1 ;  
unsigned int b = 25 ;  
char c = 'X';  
scanf ("%d%u%c",&a,&b,&c);
```

Remarque:

- On peut placer la longueur de la variable entre le signe % et la lettre spécificateur. Par exemple « %3d » indique qu'on va lire un entier de 3 chiffres.
- Si l'on sépare les spécificateurs de format par des espaces ou par des virgules alors les valeurs à lire seront séparées par les mêmes séparateurs.

#### 1.2. La fonction gets

Elle renvoie une chaîne de caractères lue dans le flux en entrée stdin. Sa déclaration est la suivante :

```
Char * gets (char * s) ;
```

Lorsqu'on lit une chaîne de caractères avec scanf, la lecture s'arrête dès la rencontre d'un blanc. Avec la fonction gets, la lecture se termine à la réception d'un retour chariot '\n'.

Exemple :

```
char *CH ;  
gets (CH) ;
```

### 1.3. Les fonctions getch(), getche() et getchar()

La première fonction renvoie un caractère lu au clavier sans écho à l'écran alors que la deuxième renvoie un caractère lu au clavier avec écho à l'écran. La fonction getchar renvoie un caractère depuis stdin.

## 2. Les fonctions de sorties

### 2.1. La fonction printf

Elle permet la traduction de quantité alphanumérique en chaîne de caractères ainsi qu'une présentation formatée des éditions.

```
int printf (const char *format, liste d'expressions) ;
```

**format** : format de représentation.

**liste d'expressions** : variables et expressions dont les valeurs sont à éditer.

La première partie est en fait une chaîne de caractères qui peut contenir

- du texte
- des séquences d'échappement qui permettent de contrôler l'affichage des données.  
Exemple '\n' qui permet le passage à la ligne suivante
- des spécificateurs de format ou de conversion

Elle contient exactement un spécificateur de format pour chaque expression.

Exemple :

```
int i ; int j = 5 ;  
printf("entrer une valeur:\n") ;  
printf("la valeur de j est :%d", j)
```

### 2.2. Les fonctions puts, putchar

La fonction puts envoie une chaîne de caractères vers stdout et ajoute un saut de ligne (newline).

```
int puts (const char *s) ;
```

Exemple :

```
puts ("ceci est un exemple") ;
```

La fonction putchar envoie un caractère vers stdout (écran)

```
int putchar (int c) ;
```

Exemple :

```
char c = 'A';  
putchar (c) ;  
putchar ('B');
```

## Partie 3 : Les instructions de branchement conditionnel

### 1. If – else

```
if ( expression )
    bloc d'instructions 1
else
    bloc d'instructions 2
```

- Si l'**expression** fournit une valeur différente de zéro, alors le **bloc d'instructions 1** est exécuté
- Si l' **expression** fournit la valeur zéro, alors le **bloc d'instructions 2** est exécuté

La partie **expression** peut désigner :

- une variable d'un type numérique,
- une expression fournissant un résultat numérique.

La partie **bloc d'instructions** peut désigner :

- un (vrai) bloc d'instructions compris entre accolades.
- une seule instruction terminée par un point-virgule.

On peut avoir des « **if** » imbriqués.

Exemples:

```
if (a > b)
    max = a ;
else
    max = b;
if (temperature < 0)
{ printf ("glace");
}
else
{   if (temperature < 100)
    { printf ("eau");
    }
    else
    { printf ("vapeur ");
    }
}
```

### 2. Switch

C'est un moyen qui permet de décider dans une structure à plusieurs cas. Cette instruction teste si une expression prend une valeur parmi un ensemble de constante et fait le branchement en conséquence.

**Switch (expression)**

```
{  
    case expr_cst1 : liste d'instructions 1  
    case expr_cst2 : liste d'instructions 2  
    ...  
    default : liste d'instructions N  
}
```

- Le choix de l'ensemble d'instructions à exécuter est calculé par l'évaluation de **expression** qui doit envoyer un type entier.
- **expr\_cste** doit être un **int** et doit être unique.

Exemple :

```
int mois ;  
scanf ("%d", &mois) ;  
switch (mois)  
{  
    case 1 : printf (janvier) ; break ;  
    case 2 : printf (fevrier) ; break ;  
    ...  
    case 12 : printf (décembre) ; break ;  
    default : printf (erreur) ;  
}
```

L'instruction **break** permet de sortir de l'instruction **switch**. Elle est importante car si on ne la met pas après chaque cas d'exécution alors toutes les instructions après ce cas seront exécutées (bien sûr s'ils ne sont pas suivis d'une autre instruction **break**).

Remarque :

Il existe des instructions de branchement non conditionnels : **goto**, **continue** et **break**.

## Partie 4 : Les structures répétitives

### 1. Introduction

En C, nous disposons de trois types d'instructions qui nous permettent la définition de boucles conditionnelles:

- la boucle : **while**
- la boucle : **do - while**
- la boucle : **for**

Théoriquement, ces structures sont interchangeables, c.à.d qu'il serait possible de programmer toutes sortes de boucles conditionnelles en n'utilisant qu'un seul type de boucle. Mais il est recommandé de choisir toujours la structure la mieux adaptée au cas du traitement à faire.

### 2. Les instructions répétitives

#### 2.1. While

|                                                  |
|--------------------------------------------------|
| <b>while</b> (expression)<br>Bloc d'instructions |
|--------------------------------------------------|

Tant que l'expression est vraie, le bloc d'instructions est exécuté. Sinon, sortir de la boucle.

Exemple 1:

|                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------|
| <pre>/* Afficher les nombres de 0 à 9 */ int i = 0 ; while (i &lt; 10) {   printf("%d \n", i) ;     i++ ; }</pre> |
|-------------------------------------------------------------------------------------------------------------------|

#### 2.2. do-While

|                                                                |
|----------------------------------------------------------------|
| <b>Do</b><br>bloc d'instructions<br><b>while</b> (expression); |
|----------------------------------------------------------------|

Le bloc d'instructions est exécuté au moins une fois et aussi longtemps que l'expression fournit une valeur différente de zéro.

Une application typique de do - while est la saisie de données qui doivent remplir une certaine condition.

Exemple:

|                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>float N ; do {   printf (« Introduisez un nombre entre 1 et 10 : ») ;     scanf (« %f », &amp;N) ; } while (N&lt;1    N &gt; 10) ;</pre> |
|-----------------------------------------------------------------------------------------------------------------------------------------------|

### 2.3. For

La structure pour en langage algorithmique est utilisée pour faciliter la programmation de boucles de comptage. La structure for en C est plus générale et beaucoup plus puissante.

```
for (expr1 ; expr2 ; expr3)  
    bloc d'instructions
```

Est équivalente à :

```
expr1 ;  
while (expr2)  
{ bloc d'instructions  
    expr3 ;  
}
```

**expr1** est évaluée une fois avant le passage de la boucle. Elle est utilisée pour initialiser les données de la boucle.

**expr2** est évaluée avant chaque passage de la boucle. Elle est utilisée pour décider si la boucle est répétée ou non.

**expr3** est évaluée à la fin de chaque passage de la boucle. Elle est utilisée pour réinitialiser les données de la boucle.

En pratique, les parties **expr1** et **expr2** contiennent souvent plusieurs initialisations ou réinitialisations, séparées par des virgules.

Exemple1:

```
/* affichage des carrés des nombres entiers compris entre 0 et 20 */  
int i ;  
for (i = 0 ; i <= 20 ; i++)  
    printf(« le carré de %d est %d »\n », i, i*i) ;
```

## Partie 5 : Les Tableaux

### 1. Les tableaux à une dimension

#### 1.1. Déclaration et mémorisation

- Déclaration

`TypeSimple NomTableau[Dimension] ;`

Les noms des tableaux sont des identificateurs.

#### Exemples :

```
int Notes [8] ; /* déclaration d'un tableau nommé Notes, de type int et de dimension 8 */
char Tab [100] ; /* déclaration d'un tableau nommé Tab, de type char et de dimension 100 */
float moy [40] ; /* déclaration d'un tableau nommé my, de type float et de dimension 100 */
```

#### Mémorisation

En C, le nom d'un tableau est le représentant de **l'adresse du premier élément** du tableau. Les adresses des autres composantes sont calculées (automatiquement) relativement à cette adresse.

#### Exemple:

`char A[5] = {'A', 66, 70, 'C', 240} ;`

|       |      |    |    |    |     |       |
|-------|------|----|----|----|-----|-------|
| ..... | 65   | 66 | 70 | 67 | 240 | ..... |
|       | 1E02 |    |    |    |     |       |
|       | A    |    |    |    |     |       |

Si un tableau est formé de N composantes et si une composante a besoin de M octets en mémoire, alors le tableau occupera N\*M octets.

#### 1.2. Initialisation et réservation automatique

Lors de la déclaration d'un tableau, on peut initialiser les composantes du tableau, en indiquant la liste des valeurs respectives entre accolades.

#### Exemple :

`int A[5] = {10, 20, 30, 40, 50};`

Il faut évidemment veiller à ce que le nombre de valeurs dans la liste corresponde à la dimension du tableau. Si la liste ne contient pas assez de valeurs pour toutes les composantes, les composantes restantes sont initialisées à zéro.

Si la dimension n'est pas indiquée explicitement lors de l'initialisation, alors l'ordinateur réserve automatiquement le nombre d'octets nécessaires.

Exemple :

```
int A[] = {10, 20, 30, 40, 50};
```

==> réservation de 5\*sizeof(int) octets (dans notre cas: 10 octets)

### 1.3. Accès aux composantes

En déclarant un tableau par **int A[5]**; nous avons défini un tableau A avec cinq composantes, auxquelles on peut accéder par: A[0], A[1], ... , A[4]

Exemple :

```
short A[5] = {1200, 2300, 3400, 4500, 5600};
```

|            |             |             |             |             |             |
|------------|-------------|-------------|-------------|-------------|-------------|
| <b>A :</b> | <b>1200</b> | <b>2300</b> | <b>3400</b> | <b>4500</b> | <b>5600</b> |
|            | <b>A[0]</b> | <b>A[1]</b> | <b>A[2]</b> | <b>A[3]</b> | <b>A[4]</b> |

### 1.4. Affichage et affectation

La structure for se prête particulièrement bien au travail avec les tableaux. La plupart des applications se laissent implémenter par simple modification des exemples-types de l'affichage et de l'affectation.

- **Affichage du contenu d'un tableau :**

Exemple :

```
main()
{ int A[5]; int i;
  for (i = 0 ; i<5 ; i++)
    printf(« %d »,A[i]) ;
  printf(« \n »);
}
```

- **Affectation avec des valeurs provenant de l'extérieur :**

Exemple :

```
main()
{ int A[5]; int i;
  for (i = 0 ; i<5 ; i++)
    scanf(« %d »,&A[i]) ;
}
```

## 2. Les tableaux à deux dimensions

### 2.1. Déclaration et mémorisation

#### Déclaration

TypeSimple NomTableau [DimLigne][DimCol] ;

#### Exemples :

```
double A[2][5] ; /* déclaration d'un tableau à deux dimensions nommé A et de type double */
char B[4][2] ; /* déclaration d'un tableau à deux dimensions nommé B et de type char */
```

#### Mémorisation et initialisation

Comme pour les tableaux à une dimension, le nom d'un tableau est le représentant de **l'adresse du premier élément** du tableau (c.-à-d. l'adresse de la première **ligne** du tableau). Les composantes d'un tableau à deux dimensions sont stockées ligne par ligne dans la mémoire.

#### Exemple: Mémorisation d'un tableau à deux dimensions

```
short A[2][3] = { {1, 2, 6},
                  {10, 20, 54} } ;
```

### 2.2. Accès aux composantes

L'accès à un élément d'un tableau à deux dimensions se fait selon le schéma suivant :

NomTableau[Ligne][Colonne]

### 2.3. Affichage et affectation

Lors du travail avec les tableaux à deux dimensions, nous utiliserons deux indices (p.ex: I et J), et la structure **for**, souvent imbriquée, pour parcourir les lignes et les colonnes des tableaux.

#### ▪ Affichage du contenu d'un tableau à deux dimensions

```
main ()
{ int A[5][10] ;
  int i, j ;
  for (i=0 ;i<5 ;i++)
  { for (j=0 ;j<10 ;j++)
    printf(« %7d »,A[i][j]) ;
    /* Retour à la ligne */
    printf(« \n ») ;
  }
}
```

## Partie 6 : Les chaînes de caractères

**NB :** Il n'existe pas de type spécial *chaîne* ou *string* en C. Une chaîne de caractères est traitée comme un *tableau à une dimension de caractères*.

### 1. Déclaration

La déclaration d'une chaîne de caractères se fait par l'une des méthodes suivantes :

- ✓ **char NomChaine [Longueur];**
- ✓ **char \* NomChaine ;**

Exemple :

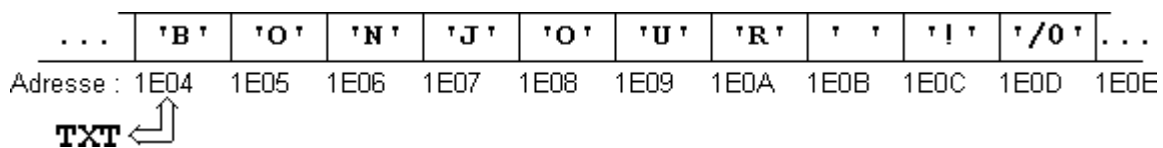
```
char NOM [20] ;    /*déclarartion sans initialisation */
char Prenom [] = « ALI » ; /* déclaration avec initialisation */
```

### 2. Mémorisation

Le nom d'une chaîne est le représentant de l'adresse du premier caractère de la chaîne. Pour mémoriser une variable qui doit être capable de contenir un texte de N caractères, nous avons besoin de N+1 octets en mémoire:

Exemple :

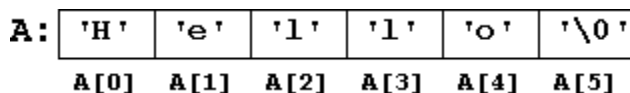
```
char TXT [10] = « BONJOUR !» ;
```



### 3. Accès aux éléments

Exemple :

```
char TXT [6] = « Hello» ;
```



### 4. Utilisation des chaînes de caractères

Les bibliothèques de fonctions de C contiennent une série de fonctions spéciales pour le traitement de chaînes de caractères.

Exemples 1: les fonctions de stdio.h

```
char TXT [6] = « Hello» ;
char CH[5] ;
printf(« %s »,TXT) ; /* affiche hello sans retour à la ligne*/
printf(« %s »,CH) ; /* CH contient l'adresse du premier caractère de la chaîne */
puts(TXT) ;          /* affiche Hello avec un retour à la ligne */
puts(« bonjour ») ; /*affiche bonjour avec un retour à la ligne */
gets(CH) ;           /* lit une ligne de caractères de stdin
```

## Exemples 2: les fonctions de string.h

```
char CH1 [] = « hello » ;  
char CH2 [] = « bonjour » ;  
char * chaine = « bonsoir » ;  
int k=2 ;  
char c='a' ;  
strlen (CH1) ; /* fournit la longueur de CH1 */  
strcpy (CH1, CH2) ; /* copie CH1 dans CH2 */  
strcat (CH1, CH2) ; /*ajoute CH2 à la fin de CH1 */  
strcmp (CH1, CH2) ; /* compare CH1 et CH2 lexico graphiquement */  
....
```

strcmp renvoie un nombre:

- positif si la chaîne1 est supérieure à la chaîne2 (au sens de l'ordre lexicographique)
- négatif si la chaîne1 est inférieure à la chaîne2
- nul si les chaînes sont identiques.

### Remarque:

Comme le nom d'une chaîne de caractères représente une adresse fixe en mémoire, on ne peut pas 'affecter' une autre chaîne au nom d'un tableau. Il faut bien copier la chaîne caractère par caractère ou utiliser la fonction **strcpy** respectivement **strncpy**:

```
strcpy (CH1, "bonsoir") ;
```

## Exemples 3: les fonctions de stlib.h

La bibliothèque <stdlib> contient des déclarations de fonctions pour la conversion de nombres en chaînes de caractères et vice-versa.

```
char CH [] = « 125 » ;  
atoi (CH) ; /* retourne la valeur numérique par CH comme int */  
atol (CH) ; /* retourne la valeur numérique représenté par CH comme long */  
atof (CH) ; /* retourne la valeur numérique représenté par CH comme double */
```

### **Règles générales pour la conversion:**

- ✓ Les espaces au début d'une chaîne sont ignorés.
- ✓ Pour une chaîne non convertible, les fonctions retournent zéro.

## Exemples 3: les fonctions de ctype.h

Les fonctions de <ctype> servent à classer et à convertir des caractères.

Dans la suite, c représente une valeur du type **int** qui peut être représentée comme caractère.

Les fonctions de **conversion** suivantes fournissent une valeur du type **int** qui peut être

```
isupper (c) ; /* retourne une valeur différente de 0 si c est une majuscule */  
islower (c) ; /*retourne une valeur différente de 0 si c est une miniscule */  
isspace (c) ; /* retourne une valeur différente de 0 si c est un signe d'espacement */  
représentée comme caractère; la valeur originale de c reste inchangée:
```

```
tolower (c) ; /* retourne c converti en minuscule si c est une majuscule */  
toupper (c) ; /* retourne c converti en majuscule si c est une minuscule */
```

## Partie 7 : Les Fonctions

### 1. Introduction

La structuration de programmes en sous-programmes se fait en C à l'aide de fonctions. Les fonctions en C correspondent aux fonctions et procédures en langage algorithmique. Nous avons déjà utilisé des fonctions prédéfinies dans des bibliothèques standards (printf, scanf,...).

### 2. Définition de fonctions

Définition d'une fonction en ANCI-C :

```
TypeRésultatRetourné NomFonction (TypePar1 NomPar1, TypeParN NomParN,...)
{
    déclarations locales
    instructions
}
```

Exemple :

```
int somme(int i, int j) /* noms et types des paramètres formels */
{
    int R ; /* déclaration locale */
    R = i + j ;
    Return (R) ; /* valeur retournée, de type int */
}
```

Remarques :

- Si nous choisissons un nom de fonction qui existe déjà dans une bibliothèque, notre fonction cache la fonction prédéfinie.
- Une fonction peut fournir comme résultat :
  - **void** (la fonction correspond alors à une 'procédure')
- Une fonction **ne peut pas** fournir comme résultat des tableaux, des chaînes de caractères. Mais, il est cependant possible de renvoyer un pointeur sur le premier élément d'un tableau ou d'une chaîne de caractères.
- Si une fonction ne fournit pas de résultat, il faut indiquer **void** (vide) comme type du résultat.
- Si une fonction n'a pas de paramètres, on peut déclarer la liste des paramètres comme **(void)** ou simplement comme **()**.
- En principe, l'ordre des définitions dans le texte du programme ne joue pas de rôle, mais chaque fonction doit être déclarée ou définie avant d'être appelée.

### 3. Déclaration de fonctions

En C, il faut déclarer chaque fonction avant de pouvoir l'utiliser. La déclaration informe le compilateur du type des paramètres et du résultat de la fonction.

La déclaration d'une fonction se fait par un **prototype** de la fonction qui indique uniquement le type des données transmises et reçues par la fonction.

TypeRésultat NomFonction (TypePar1, TypePar2, ...);Ou

bien

TypeRésultat NomFonction (TypePar1 NomPar1, TypePar NomPar2, ...);

#### Remarques :

- **Noms des paramètres** : On peut facultativement inclure les *noms des paramètres* dans la déclaration.
- **Règles pour la déclaration des fonctions** : De façon analogue aux déclarations de variables, nous pouvons déclarer une fonction localement ou globalement. La définition des fonctions joue un rôle spécial pour la déclaration. En résumé, nous allons considérer les règles suivantes:
  - **Déclaration locale**: Une fonction peut être déclarée localement *dans la fonction qui l'appelle* (avant la déclaration des variables). Elle est alors disponible à cette fonction.
  - **Déclaration globale**: Une fonction peut être déclarée globalement *au début du programme* (derrière les instructions **#include**). Elle est alors disponible à toutes les fonctions du programme.

### 4. Paramètres d'une fonction

Les paramètres d'une fonction sont simplement des variables locales qui sont initialisées par les valeurs obtenues lors de l'appel. Lors d'un appel, le nombre et l'ordre des paramètres doivent nécessairement correspondre aux indications de la déclaration de la fonction.

### 5. Passage des paramètres par valeur

Les paramètres d'une fonction sont à considérer comme des **variables locales** qui sont initialisées automatiquement par les valeurs indiquées lors d'un appel. A l'intérieur de la fonction, nous pouvons donc changer les valeurs des paramètres sans influencer les valeurs originales dans les fonctions appelantes.

Exemple : la fonction somme(int, int) listé précédemment.

### 6. Passage de l'adresse d'une variable (Passage par adresse)

Comme nous l'avons constaté ci-dessus, une fonction n'obtient que les valeurs de ses paramètres. Pour changer la valeur d'une variable de la fonction appelante, il faut procéder comme suit:

- la fonction appelante doit *fournir l'adresse de la variable* et
- la fonction appelée doit *déclarer le paramètre comme pointeur*.

Exemple :

```
int somme(int *i, int *j)
{   int R ; /* déclaration locale */
    R = *i + *j ;
    Return (R) ; /* valeur retournée, de type int */
}
main()
{   int a = 5, b = 9, S ;
    S = somme(&a, &b); /* appel de la fonction */
}
```

## Partie 8 : Les Structures

### 1. Introduction

Le tableau permet de désigner sous un seul nom un ensemble de valeurs de même type, repérées par des indices.

Un enregistrement, appelé structure en langage C, est une variable complexe qui permet de désigner sous un seul nom un ensemble de valeurs pouvant être de types différents.

Nom du champ :

Chaque élément de la structure est nommé : champ

L'accès à un champ se fait par son nom dans la structure.

### 2. Déclaration d'une structure

Lors de la déclaration de la structure, on définit un modèle où on indique les champs de la structure, c'est-à-dire le type et le nom des variables qui la composent :

Syntaxe:

```
Struct <Nom_Structure>
{ <type_champ1> <Nom_Champ1>;
  <type_champ2> <Nom_Champ2>;
  <type_champ3> <Nom_Champ3>;
  ...
};
```

Exemple:

```
struct Etudiant
{ char Nom[30];
  int Age;
  float MoyenneScolaire;
};
```

REMARQUES :

- Le nom des champs répond aux critères des noms de variables.
- Deux champs ne peuvent avoir le même nom.
- Les données peuvent être de n'importe quel type hormis le type de la structure dans laquelle elles se trouvent.

### 3. Définition d'une variable de type structure

La définition d'une variable structurée est une opération qui consiste à créer une variable ayant comme type celui d'une structure que l'on a précédemment déclaré, c'est-à-dire la nommer et lui réserver un emplacement en mémoire.

Une variable structurée doit être définie comme suit :

```
struct <Nom_Structure> <Var_Structure>;
```

Exemples:

```
struct Etudiant
{
    char Nom[12];
    int Age;
    float MoyenneScolaire;
};
struct Etudiant E1, E2;
Ou bien
struct Etudiant
{
    char Nom[12];
    int Age;
    char Sexe;
    float MoyenneScolaire;
}E1, E2;
```

### 4. Initialisation d'une structure

Lors de sa déclaration, on peut initialiser une structure en indiquant entre {} la liste des valeurs séparées par des virgules.

Chaque valeur étant une constante ayant le type du champ correspondant.

Exemples:

```
struct Etudiant
{
    int niveau;
    float moyenne;
};
struct Etudiant E1 = {2, 10.9};
struct Etudiant E2 = {2, 12.5};
```

### 5. Utilisation d'une structure

Chaque variable de type structure possède des champs repérés avec des noms uniques.

Toutefois le nom des champs ne suffit pas pour y accéder étant donné qu'ils n'ont de contexte qu'au sein de la variable structurée...

Pour accéder aux champs d'une structure on utilise l'opérateur de champ (un simple point) placé entre le nom de la variable structurée et le nom du champ.

Syntaxe:

<Var\_Structure>.<Nom\_Champi>;

Ainsi, pour affecter des valeurs à la variable E1, on pourra écrire:

```
E1.niveau = 2;
scanf ("%f", &E1.moyenne) ;
printf ("Moyenne = %f", E1.moyenne) ;
```

Il est clair, qu'on peut effectuer des affectations entre deux variables de même type de structure :

- soit d'une façon individuelle (champs par champs), sur chacun de leurs champs,
- soit d'une manière globale sur toute la structure.

Exemple1: Affectation individuelle :

```
Etudiant1.niveau = Etudiant2.niveau;
Etudiant1.moyenne = Etudiant2.moyenne;
```

Exemple2: Affectation Globale :

```
Etudiant1 = Etudiant2;
```

Exercice d'application 1: Définir une fiche Etudiant. Saisir dans le programme les informations d'un étudiant et les affichées.  
Informations: Nom, prénom, age, moyenne.

## 6. Structure comportant des tableaux

Comme on a vu précédemment, une structure peut contenir des champs de type chaîne de caractères ou bien de type tableau.

Exemple:

```
struct UnEtudiant
{
    char nom [30];
    char prenom [30];
    float notes [4];
    int niveau;
    float moyenne;
};
struct UnEtudiant Etudiant1, Etudiant2;
```

Notation:

*Etudiant1.nom[0] : désigne le premier caractère du champ nom de l'étudiant Etudiant1.*  
*Etudiant2.notes[3] : désigne la quatrième note du tableau notes de l'étudiant Etudiant2*

Initialisation:

```
struct UnEtudiant Etudiant1 = {"Ben Flen", "Ali", {10.0, 12.5, 13.6, 7.5}, 2, 10.9};
struct UnEtudiant Etudiant2 = {"Kebbas ", "Ammar", {12.0, 12.5, 14.0, 11.5}, 2, 12.5};
```

### Utilisation:

```
strcpy (Etudiant1.nom, "Ben Flen");  
Etudiant1.notes[2] = 13.6;  
printf ("%f", Etudiant2.notes[i]);
```

## **7. Structure comportant d'autres structures**

Une structure peut contenir aussi des champs de type structure. Cette dernière doit être différente de la première.

### Exemple:

```
struct Date  
{  
    int jour;  
    int mois;  
    int annee;  
};  
struct UnEtudiant  
{  
    char nom [30] ;  
    char prenom [30] ;  
    struct Date date_naissance ;  
};
```

### Initialisation:

```
struct UnEtudiant Eudiant1 = {"Ali", "Ben Flen", {1, 1, 2000}};
```

### Utilisation:

```
Etudiant1.date.jour = 1;  
scanf ("%d %d %d", &Etudiant1.date.jour, &Etudiant1.date.mois, &Etudiant1.date.annee);
```

## **8. Tableaux de structures**

Étant donné qu'une structure est composée d'éléments de taille fixes, il est possible de créer un tableau ne contenant que des éléments du type d'une structure donnée.

Il suffit de créer un tableau dont le type est celui de la structure et de le repérer par un nom de variable:

### Déclaration:

```
struct <Nom_Structure>  
{ <type_champ1> <Nom_Champ1>;  
  ... <type_champN> <Nom_ChampN>;  
};  
struct <Nom_Structure> <Tab_Struct>[N];
```

Accès: A un élément du tableau : `<Tab_Struct>[i]` (structure N° i)

A un élément d'une structure : `<TabStruct>[i].<NomChampj>`

Exemple: Soit la déclaration suivante :

```
struct Etudiant
{
    char nom [30] ;
    char prenom [30] ;
    float notes [4] ;
};
struct Etudiant tab [50];
```

On peut écrire les instructions suivantes :

```
for (i = 0; i < 50; i++)
{
    printf("Entrez le nom de l'étudiant n°%d ", i+1);
    scanf("%s", tab[i].nom);
    printf("Entrez le prénom de l'étudiant n°%d ", i+1);
    scanf("%s", tab[i].prenom);
    for (j = 0; j < 4; j++)
    {
        printf("Entrez la note %d de l'étudiant n°%d ", j+1, i+1);
        scanf("%f", &tab[i].notes[j]);
    }
}
```

## 9. Utilisation de « typedef »

La principale utilité des typedef est de faciliter l'écriture des programmes. Lorsqu'on donne un nom à un type structure par typedef, l'utilisation est beaucoup plus aisée. En effet, si on déclare :

```
struct Pers { ... };
```

les déclarations de variables se feront par :

```
struct Pers P1,P2;
```

Alors que si on déclare :

```
typedef struct Pers { ... };
```

les déclarations de variables se feront par :

```
struct Pers P1,P2;
```

ou bien par:

```
Personne P1,P2;
```

On voit que la seconde méthode permet d'éviter la répétition du mot clé *struct*.

Remarque:

On peut aussi faire la déclaration comme suit: Dans ce cas, les déclarations de variables se feront uniquement par :

```
Personne P1,P2;
```

```
typedef struct { ... } Personne;
```

```
struct Pers { ... };
```

```
typedef struct Pers Personne;
```

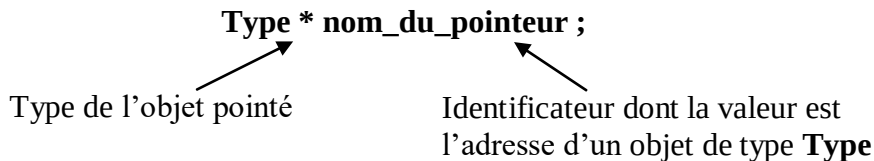
## Partie 9 : Pointeurs et Allocation dynamique

### 1. Introduction

Toute variable manipulée dans un programme est stockée quelque part en mémoire centrale. Cette mémoire est constituée de mots qui sont identifiés de manière univoque par un numéro qu'on appelle adresse. Pour retrouver une variable, il suffit donc de connaître l'adresse du mot où elle est stockée ou (s'il s'agit d'une variable qui recouvre plusieurs mots contigus) l'adresse du premier de ces mots. Pour des raisons évidentes de lisibilité, on désigne souvent les variables par des identificateurs et non par leurs adresses. C'est le compilateur qui fait alors le lien entre l'identificateur d'une variable et son adresse en mémoire. Toutefois, il est parfois très pratique de manipuler directement une variable par son adresse.

### 2. Notion de pointeur

Un pointeur est un objet (variable) dont la valeur est égale à l'adresse d'un autre objet. On déclare un pointeur par l'instruction :



Exemple :

```
char * pc ; /* pc est un pointeur pointant sur un objet de type char */  
int * pi ; /* pi est un pointeur pointant sur un objet de type int */
```

L'opérateur unaire d'indirection \* permet d'accéder directement à la valeur de l'objet pointé. Ainsi si p est un pointeur vers un entier i, \*p désigne la valeur de i. Par exemple, le programme :

```
main()  
{ int i = 3 ;  
  int * p ; /* p est un pointeur sur un objet de type entier */  
  p = &i ; /* p contient l'adresse de la variable */  
  printf(« contenu de la case mémoire pointée par p est : %d », *p) ;  
}
```

Imprime : contenu de la case mémoire pointée par p est : 3

Dans ce programme, les objets **i** et **\*p** sont identiques. Ils ont même adresse et valeur. Cela signifie en particulier que toute modification de \*p modifie i.

Remarque :

- La valeur d'un pointeur est toujours un entier.
- Le type d'un pointeur dépend du type de la variable à laquelle il pointe.

### 3.1.L'opérateur « adresse de » : &

Pour avoir l'adresse mémoire d'une variable, il faudrait utiliser l'opérateur « adresse de »

: &Ainsi :

&<Nom\_Variable> : fournit l'adresse de la variable <Nom\_Variable>

#### Exemple:

```
int A = 2;
char B;
int *p1;
char *p2;
p1 = &A; /* p1 pointe sur A */
```

### 3.2.L'opérateur « contenu de » : \*

Pour accéder au contenu de l'adresse mémoire pointée par un pointeur, il faudrait utiliser l'opérateur « contenu de » :

\*Ainsi :

\*<Nom\_Pointeur> : désigne le contenu de l'adresse référencée par le pointeur <Nom\_Pointeur>

#### Exemple:

```
int A=10, B=50;
int *P;
P = &A; /* P pointe sur A */
B = *P; /* B contient 10 */
P = 20; /* A contient 20 */
```

## 3. Arithmétique des pointeurs

La valeur d'un pointeur étant un entier. On peut lui appliquer un certain nombre d'opérateurs arithmétiques classiques. Les seules opérations arithmétiques valides sur les pointeurs sont :

- l'addition d'un entier à un pointeur. Le résultat est un pointeur de même type que le pointeur ledépart.
- la soustraction d'un entier à un pointeur. Le résultat est un pointeur de même type que lepointeur de départ.
- la différence de deux pointeurs p et q pointant tous deux vers des objets de même type. Lerésultat est un entier dont la valeur est égale à **(p-q)/sizeof(type)**.

**NB : Notons que la somme de deux pointeurs n'est pas autorisée.**

#### Remarque :

Les opérateurs de comparaison sont également applicables aux pointeurs à condition de comparer des pointeurs qui pointent vers des objets de même type.

#### Exemples:

```

int *pi; /* pi pointe sur un objet de type entier */
float *pr; /* pr pointe sur un objet de type réel */
char *pc; /* pc pointe sur un objet de type caractère */
*pi = 421; /* 421 est le contenu de la case mémoire pointée par pi et des 3 suivantes */
*(pi+1) = 53; /* on range 53 4 cases mémoire plus loin */
*pr = 45.7; /* 45.7 est rangé dans la case mémoire pointée par pr et les 3 suivantes */
pr++; /* incrémente la valeur du pointeur pr (de 4 cases mémoire) */
*pc = 'j'; /* le contenu de la case mémoire pc est le code ASCII de 'j' */
pc--; /* décrémente la valeur du pointeur c (d'une case mémoire) */

```

## 4. Allocation dynamique

Lorsque l'on déclare une variable char, int, float .... Un nombre de cases mémoire bien défini est **réservé** pour cette variable. Il n'en est pas de même avec les pointeurs. Avant de manipuler un pointeur, il faut l'**initialiser soit par une allocation dynamique soit par une affectation d'adresse p=&i**. Sinon, par défaut la valeur du pointeur est égale à une constante symbolique notée NULL définie dans « stdio.h ».

On peut initialiser un pointeur **p** par une affectation sur **p**. Mais, il faut d'abord réserver à **p** un espace mémoire de taille adéquate. L'adresse de cet espace mémoire sera la valeur de **p**. Cette opération consistant à réserver un espace mémoire pour stocker l'objet pointé s'appelle **allocation dynamique**. Elle se fait en C par la fonction **malloc** de la librairie standard **stdlib.h**. Sa syntaxe est la suivante :

```
malloc (nombre_cases_mémoire)
```

Cette fonction retourne un pointeur de type char\* pointant vers un objet de taille nombre\_cases\_mémoire. Pour initialiser des pointeurs vers des objets qui ne sont pas de type char\*, il faut convertir le type de la sortie de la fonction malloc à l'aide d'un cast.

### Exemples : la fonction malloc

```

char *pc ;
int *pi, *pj, *pk ;
float *pr ;
pc = (char*)malloc(10) ; /* on réserve 10 cases mémoire, la place pour 10 caractères */
pi = (int*)malloc(16) ; /* on réserve 16 cases mémoire, soit la place pour 4 entiers */ pr
pr = (float*)malloc(24) ; /* on réserve 24 cases mémoire, soit la place pour 6 réels */
pj = (int*)malloc(sizeof(int)) ; /* on réserve la taille d'un entier en mémoire */
pk = (int*)malloc(3*sizeof(int)) ; /* on réserve la place en mémoire pour 3 entiers */

```

### Remarque1 :

Dans l'exemple:

```

main()
{ int i ;
  int *p ;
  p = &i ;
}

```

i et \*p sont identiques et on n'a pas besoin d'allocation dynamique puisque l'espace mémoire à l'adresse &i est déjà réservé pour un entier.

### Libération de la mémoire : la fonction free

La libération d'un pointeur se fait en langage C grâce à la fonction free(). La déclaration de cette fonction se trouve également dans le fichier en-tête stdlib.h.

```
pi = (int*)malloc(16); /* on réserve 16 cases mémoire, soit la place pour 4 entiers */
pr = (float*)malloc(24); /* on réserve 24 cases mémoire, soit la place pour 6 réels */
free(pi); /* on libère la place précédemment réservée pour pi */
```

## 5. Pointeurs et tableaux

Tout tableau en C est en fait un pointeur constant. Dans la déclaration :

```
int tab[10];
```

**tab** est un pointeur constant non modifiable dont la valeur est l'adresse du premier élément du tableau. Autrement dit **tab** a pour valeur **&tab[0]**. On peut donc utiliser un pointeur initialisé à tab pour parcourir les éléments du tableau.

Exemple :

```
main()
{ int tab[5] = {2, 10, 5, 89, 0};
  int *p;
  p = tab; /* ou p = &tab[0]; */
  for(int i=0; i<5; i++)
  { printf("%d\n", *p);
    p++; } }
```

## 6. Pointeurs sur structures

Il est possible de déclarer un pointeur sur une variable de type structure et de l'utiliser comme tout autre pointeur.

Le langage C a prévu un nouvel opérateur, noté ->:

(\*x).y sera exprimé plus clairement par: x->y.

Exemple :

```
#include <stdio.h>
#include <stdlib.h>
struct DateDef
{   int jour;
    int mois;
    int annee;
};
typedef struct DateDef Date;
int main()
{Date *Date_ptr;
  Date_ptr = (struct Date*)malloc(sizeof(struct Date));
  Date_ptr->jour = 6;
  Date_ptr->mois = 10;
  Date_ptr->annee = 2016;
  printf("la date saisie est %d/%d/%d.\n", Date_ptr->jour,
  Date_ptr->mois, Date_ptr->annee );
  return 0;
}
```

## Partie 10 : Les listes chaînées

### 1 Définition

Une liste chaînée est constituée d'un ensemble de cellules chaînées entre elles qui contiennent les informations à mémoriser. L'emplacement mémoire nécessaire à chaque cellule est alloué dynamiquement. Dans une forme simple de liste chaînée, chaque cellule comprend l'élément de la liste à mémoriser et l'adresse de la cellule suivante. La dernière cellule contient un pointeur qui contient une adresse nulle, ce qui indique la fin de la liste. C'est l'adresse de la première cellule qui détermine la liste.

### 2 Implémentation d'une liste chaînée

On suppose dans la suite que les valeurs à mémoriser sont de type réel, mais évidemment tout autre type de données est possible. On définit un type structure cellule regroupant une valeur réelle et un pointeur.

```
struct cellule
{
    float val ;

    struct cellule * suivant;
} ;
```

On considère une liste L tel que L est de type struct cellule \* (pointeur sur une cellule).

**// Initialisation de la liste vide**

```
void initialiser_liste (struct cellule * L)
{
    L = Null ;
}
```

**// Affichage de la liste**

```
void afficher_liste (struct cellule * L)
{
    struct cellule * p ;p =
    L ;

    while (p!=NULL)
    {
        printf("%f ", p -> val);p =
        p -> suivant;
    }
    printf("\n");
}
```

**// Insertion d'une nouvelle cellule en début de liste**

```
struct cellule * inserer_premier (struct cellule * L, float vale)
{
    struct cellule * nouveau ;
    nouveau = (struct cellule *) malloc (sizeof(struct cellule)) ;

    nouveau -> val = vale;

    nouveau -> suivant = L;L = nouveau ;
```

```

Return (L) ;
}
// Insertion d'une nouvelle cellule en fin de liste

void inserer_fin (struct cellule * L, float vale)

{
struct cellule * nouveau , * parcours;
nouveau = (struct cellule *) malloc (sizeof(struct cellule)) ;

nouveau -> val = vale;

nouveau -> suivant = NULL;

if (L == NULL)

{
L= nouveau ;
}
else
{
parcours = L ;
while ( parcours -> suivant !=NULL)
{
parcours = parcours -> suivant ;
}
parcours ->suivant = nouveau ;
}
}

// Suppression d'une cellule en début de liste

float enlever_premier (struct cellule * L)

{
struct cellule * inter;

float vale ;

if (L != NULL)
{
inter = L ;
L= inter -> suivant;

vale = inter -> val ;

free(inter) ;

return (vale) ;
}
else
{
return(-1);
}
}

// Suppression d'une cellule en fin de liste

float enlever_fin (struct cellule * L)

{

```

```

struct cellule * pred, * parcours;
float vale ;
if (L != NULL)
{
    parcours = L ;

    pred = L ;

    while ( parcours -> suivant !=NULL)
    {
        pred = parcours ;
        parcours = parcours -> suivant ;
    }
    vale = parcours -> val ;if
    (parcours == L)

    {
        L= NULL ;
    }
    else
    {
        pred -> suivant = NULL;
    }
    free(parcours);

    return (vale);

}
else return (-1);
}
// Recherche d'une valeur dans la liste int

recherche (struct cellule * L, float vale)

{
    struct cellule * parcours;if
    (L == NULL)

    {
        return(0) ;
    }
    else
    {
        parcours = L ;
        while (( parcours -> suivant !=NULL) && (parcours -> val !=vale))

        {
            parcours = parcours -> suivant ;
        }
        if (parcours -> val == vale) return (1);

        else return (0);

    }
}

```

### **// Suppression d'une valeur dans la liste**

```
void supprime(struct cellule * L, float vale)
{
struct cellule * pred, * parcours;if
(L != NULL)
{
parcours = L ;
pred = L ;
while ((parcours -> suivant !=NULL)&&(parcours -> val !=vale))
{
pred = parcours ;
parcours = parcours -> suivant ;
}
if (parcours -> val == vale)
{
if (L== parcours)
L = parcours ->suivant;
else pred->suivant = parcours-> suivant ;
free (parcours) ;
}
}
}
```